НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №2

«Введение в алгоритмическую и

высокоуровневую оптимизацию на примере

задачи умножения матриц»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 22.03.2026

2026 г.

Введение

Основная цель работы:

Познакомиться с базовыми принципами выполнения алгоритмической и высокоуровневой оптимизации программного кода на примере задачи умножения квадратных матриц.

Описание решаемой задачи

Задача: Реализовать следующие варианты кода для умножения матриц размером N

1. Классическое умножение матриц;

2. Классическое умножение матриц с предварительным транспонированием матрицы **B**;

3. Умножение матриц с использованием буферизации столбца матрицы **B**;

4. Блочное умножение матриц (размер блока **S** задается как параметр).

Необходимо затем сравнить время выполнения каждого алгоритма (время **t** и перевести его в $\mathbf{P_{real}}$ – GFLOP/s) в конфигурациях запуска «Debug» без оптимизации компилятора и «Release» с оптимизацией. Провести проверку совпадения результатов работы четырёх данных алгоритмов. Сделать выводы о получаемом выигрыше в скорости обработки. Выбрать оптимальные значения $\mathbf{S^*}$ и $\mathbf{M^*}$ для блочного умножения и буферизации столбца соответственно (по максимальному $\mathbf{P_{real}}$), затем сравнить быстродействие наилучших реализаций.

Входные данные:

1) Число N = 1024 — размер квадратных матриц NxN, которые мы умножаем;

2) Числа M = 1, 2, 4, 8, 16 — степени раскрутки внутреннего цикла для алгоритмов буферизации столбца и блочного умножения;

3) Числа S = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 — размеры блока для блочного умножения.

Результат: Время выполнения **t** и реальная производительность $\mathbf{P_{real}}$, равная $\mathbf{2N^3/t}$, оценка объёма FP-вычислений (примерно N^3 умножений и N^3 сложений)

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Кэш-память CPU: L1d: 512 KiB; L1i: 512 KiB; L2: 1 MiB; L3: 8 MiB
- Размер RAM: 15Gi
- Компилятор: g++
- Сборка **Release**: g++ -std=c++17 -O3 -DNDEBUG -march=native
- Сборка **Debug** (без оптимизаций): g++ -std=c++17 -O0 -g -march=native
- Измерение времени: C++ функция — std::chrono::steady_clock

Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab02/lab02.cpp

Ниже приведён полный листинг:

```
#include <algorithm>

#include <iostream>
#include <cstddef>
#include <fstream>
#include <iomanip>
#include <random>
#include <string>
#include <vector>
#include <chrono>
#include <cmath>

using f32 = float;


static inline double now_sec() {
    using clock = std::chrono::steady_clock;

    return std::chrono::duration<double>(clock::now().time_since_epoch()).count();
}


static inline double preal_gflops(std::size_t N, double t_sec) {
    const double V = 2.0 * std::pow(double(N), 3);
```

```
    return (V / t_sec) / 1e9;
}


static void fill_random(std::vector<f32>& M, unsigned seed) {
    std::mt19937 rng(seed);
    std::uniform_real_distribution<f32> dist(-1.0f, 1.0f);

    for (auto &x : M) x = dist(rng);
}


static void zero(std::vector<f32>& C) { std::fill(C.begin(), C.end(), 0.0f); }


static bool compare_C(const std::vector<f32>& C1, const std::vector<f32>& C2, f32
rtol = 1e-4f, f32 atol = 1e-4f) {

    if (C1.size() != C2.size()) return false;

    for (std::size_t i = 0; i < C1.size(); ++i) {
        f32 a = C1[i], b = C2[i];
        f32 diff = std::fabs(a - b);
        f32 tol  = atol + rtol * std::fabs(b);
        if (diff > tol)
            return false;
    }

    return true;
}


static void mul_classic_ijk(const std::vector<f32>& A, const std::vector<f32>& B,
std::vector<f32>& C, std::size_t N) {
    zero(C);
    for (std::size_t i = 0; i < N; ++i) {
        for (std::size_t j = 0; j < N; ++j) {
            f32 s = 0.0f;
            for (std::size_t k = 0; k < N; ++k)
                s += A[i * N + k] * B[k * N + j];
            C[i * N + j] = s;
        }
    }
}


static void transpose(const std::vector<f32>& B, std::vector<f32>& BT, std::size_t
N) {
    for (std::size_t i = 0; i < N; ++i)
        for (std::size_t j = 0; j < N; ++j)
            BT[j * N + i] = B[i * N + j];
}


static void mul_transposeB(const std::vector<f32>& A, const std::vector<f32>& B,
std::vector<f32>& C, std::size_t N, double* t_transpose_sec) {
    std::vector<f32> BT(N*N);

    double t0 = now_sec();
    transpose(B, BT, N);
    double t1 = now_sec();
```

```
    if (t_transpose_sec)
        *t_transpose_sec = (t1 - t0);

    zero(C);
    for (std::size_t i = 0; i < N; ++i) {
        const f32* arow = &A[i * N];
        for (std::size_t j = 0; j < N; ++j) {
            const f32* brow = &BT[j * N];
            f32 s = 0.0f;
            for (std::size_t k = 0; k < N; ++k)
                s += arow[k] * brow[k];
            C[i * N + j] = s;
        }
    }
}


static void mul_buffer_colB_unrollM(const std::vector<f32>& A, const
std::vector<f32>& B, std::vector<f32>& C, std::size_t N, std::size_t M) {
    zero(C);
    if (M == 0)
        M = 1;

    if (M > N)
        M = N;

    std::vector<f32> tmp(N);

    for (std::size_t j = 0; j < N; ++j) {
        for (std::size_t k = 0; k < N; ++k)
            tmp[k] = B[k * N + j];

        for (std::size_t i = 0; i < N; ++i) {
            const f32* arow = &A[i * N];
            f32 s = 0.0f;

            std::size_t k = 0;
            for (; k + M <= N; k += M) {
                for (std::size_t u = 0; u < M; ++u)
                    s += arow[k + u] * tmp[k + u];
            }
            for (; k < N; ++k)
                s += arow[k] * tmp[k];

            C[i * N + j] = s;
        }
    }
}


static void mul_blocked_unrollM(const std::vector<f32>& A, const std::vector<f32>&
B, std::vector<f32>& C, std::size_t N, std::size_t S, std::size_t M) {
    zero(C);
    if (S == 0)
        S = 1;

    if (M == 0)
        M = 1;

    if (S > N)
        S = N;
```

```
    for (std::size_t ii = 0; ii < N; ii += S) {
        std::size_t i_max = std::min(ii + S, N);

        for (std::size_t jj = 0; jj < N; jj += S) {
            std::size_t j_max = std::min(jj + S, N);

            for (std::size_t kk = 0; kk < N; kk += S) {
                std::size_t k_max = std::min(kk + S, N);

                for (std::size_t i = ii; i < i_max; ++i) {
                    const f32* arow = &A[i * N];

                    for (std::size_t j = jj; j < j_max; ++j) {
                        f32 s = C[i * N + j];

                        std::size_t k = kk;

                        for (; k + M <= k_max; k += M) {
                            for (std::size_t u = 0; u < M; ++u)
                                s += arow[k + u] * B[(k + u) * N + j];
                        }

                        for (; k < k_max; ++k)
                            s += arow[k] * B[k * N + j];

                        C[i * N + j] = s;
                    }
                }
            }
        }
    }
}


template<class Fn>
static double bench_best(Fn fn, int repeats) {
    double best = 1e100;

    for (int r = 0; r < repeats; ++r) {
        double t0 = now_sec();
        fn();
        double t1 = now_sec();
        best = std::min(best, t1 - t0);
    }

    return best;
}


static void write_csv_header(std::ofstream& out) {
    out << "mode,N,S,M,algo,t_sec,preal_gflops,extra\n";
}


static void write_csv_row(std::ofstream& out, const std::string& mode, std::size_t
N, std::size_t S, std::size_t M, const std::string& algo, double t_sec, double
preal, const std::string& extra = "") {
    out << mode << "," << N << "," << S << "," << M << "," << algo << "," <<
std::setprecision(10) << t_sec << "," << std::setprecision(6) << preal << "," <<
extra << "\n";
}
```

```
static bool arg_eq(const char* a, const char* b) { return std::string(a) ==
std::string(b); }


int main(int argc, char** argv) {
    std::string mode = "single";
    std::size_t N = 512, S = 64, M = 4;
    std::string out_path = "results.csv";
    int repeats = 1;

    for (int i = 1; i < argc; ++i) {
        if (arg_eq(argv[i], "--mode") && i + 1 < argc) mode = argv[++i];
        else if (arg_eq(argv[i], "--N") && i + 1 < argc) N = std::stoull(argv[+
+i]);
        else if (arg_eq(argv[i], "--S") && i + 1 < argc) S = std::stoull(argv[+
+i]);
        else if (arg_eq(argv[i], "--M") && i + 1 < argc) M = std::stoull(argv[+
+i]);
        else if (arg_eq(argv[i], "--out") && i + 1 < argc) out_path = argv[++i];
        else if (arg_eq(argv[i], "--repeats") && i + 1 < argc) repeats =
std::stoi(argv[++i]);
    }

    std::ofstream out(out_path, std::ios::out);
    write_csv_header(out);

    auto run_single = [&](std::size_t N0, std::size_t S0, std::size_t M0, const
std::string& mode_name) {
        std::vector<f32> A(N0*N0), B(N0*N0);
        std::vector<f32> C1(N0*N0), C2(N0*N0), C3(N0*N0), C4(N0*N0);

        fill_random(A, 1);
        fill_random(B, 2);

        // classic
        double t1 = bench_best([&]{ mul_classic_ijk(A, B, C1, N0); }, repeats);
        write_csv_row(out, mode_name, N0, S0, M0, "classic", t1, preal_gflops(N0,
t1));

        // transpose B
        double t_tr = 0.0;
        double t2_total = bench_best([&]{ mul_transposeB(A, B, C2, N0, &t_tr); },
repeats);
        double t2_mul_only = t2_total - t_tr;

        write_csv_row(out, mode_name, N0, S0, M0, "transpose_total", t2_total,
preal_gflops(N0, t2_total));
        write_csv_row(out, mode_name, N0, S0, M0, "transpose_only",  t_tr,     0.0,
"P_real not applicable");
        write_csv_row(out, mode_name, N0, S0, M0, "transpose_mul_only",
t2_mul_only,
                      preal_gflops(N0, t2_mul_only));

        if (!compare_C(C1, C2)) {
            write_csv_row(out, mode_name, N0, S0, M0, "check", 0.0, 0.0, "Mismatch:
classic vs transpose");
            return;
        }

        // buffered column + unroll M
        double t3 = bench_best([&]{ mul_buffer_colB_unrollM(A, B, C3, N0, M0); },
```

```
repeats);
        write_csv_row(out, mode_name, N0, S0, M0, "buffered", t3, preal_gflops(N0,
t3));
        if (!compare_C(C1, C3)) {
            write_csv_row(out, mode_name, N0, S0, M0, "check", 0.0, 0.0, "Mismatch:
classic vs buffered");
            return;
        }

        // blocked + unroll M, block size S
        double t4 = bench_best([&]{ mul_blocked_unrollM(A, B, C4, N0, S0, M0); },
repeats);
        write_csv_row(out, mode_name, N0, S0, M0, "blocked", t4, preal_gflops(N0,
t4));
        if (!compare_C(C1, C4)) {
            write_csv_row(out, mode_name, N0, S0, M0, "check", 0.0, 0.0, "Mismatch:
classic vs blocked");
            return;
        }
    };

    if (mode == "single") {
        run_single(N, S, M, "single");
    } else if (mode == "sweepS") {
        for (std::size_t Ss = 1; Ss <= N; Ss <<= 1) {
            run_single(N, Ss, M, "sweepS");
        }
    } else if (mode == "sweepM") {
        for (std::size_t Mm = 1; Mm <= 16; Mm <<= 1) {
            run_single(N, S, Mm, "sweepM");
        }
    }

    return 0;
}
```

Результаты

1. Сравнение быстродействия классического умножения матриц в конфигурациях запуска «Debug» без оптимизации компилятора и «Release» с оптимизацией:

Запуск **Debug**:

N	S	M	algo	t_sec	preal_gflops
512	64	4	classic	0.853484764	0.314517

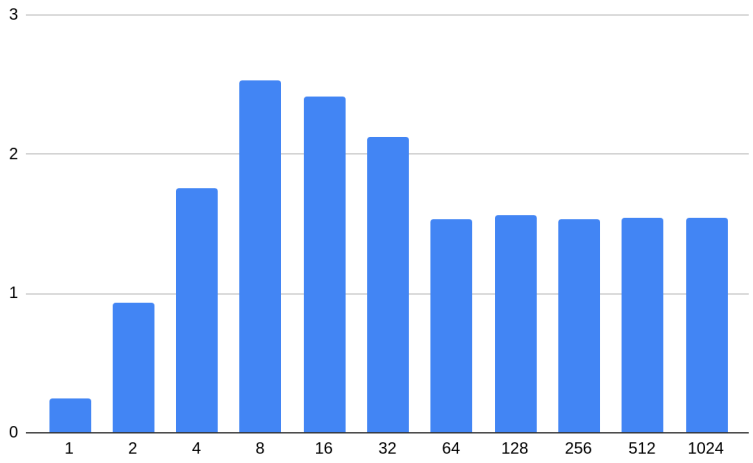
Запуск **Release**:

N	S	M	algo	t_sec	preal_gflops
512	64	4	classic	0.398955307	0.672846

Имеем выигрыш в **0.358329** GFLOP/s — то есть примерно **358 млн** FLOP в секунду! Более чем значительный выигрыш (почти в **2.14** раза), так что для анализа алгоритмических оптимизаций далее используем Release.

2. Результаты умножения для всех алгоритмов совпадают. Для этого разработана функция **compare_C**, в результате работы которой в CSV файл могло быть выведено предупреждение «Mismatch: ...», и выполнение программы останавливалось. При запуске всё сработало штатно.

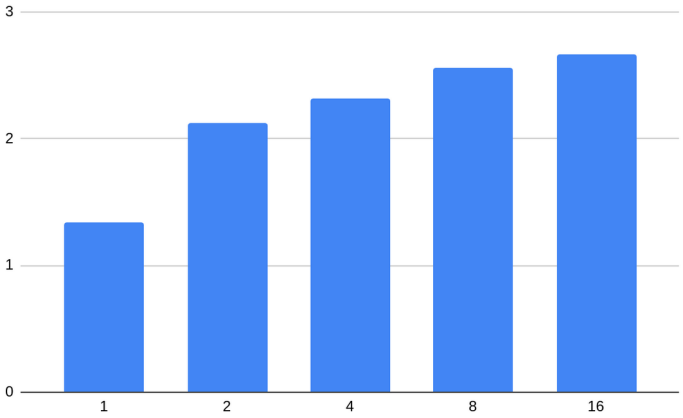
3. Вычисление P_{real} в зависимости от размера блока S, график и таблица зависимости для фиксированного N = 1024 представлены ниже.



S	t_sec (blocked)	preal_gflops
1	8.724905795	0.246133
2	2.303821433	0.93214
4	1.225656763	1.75211
8	0.848147574	2.53197
16	0.889947429	2.41305
32	1.011633471	2.12279
64	1.401813084	1.53193
128	1.375272827	1.5615
256	1.397607673	1.53654
512	1.390515439	1.54438
1024	1.394156976	1.54035

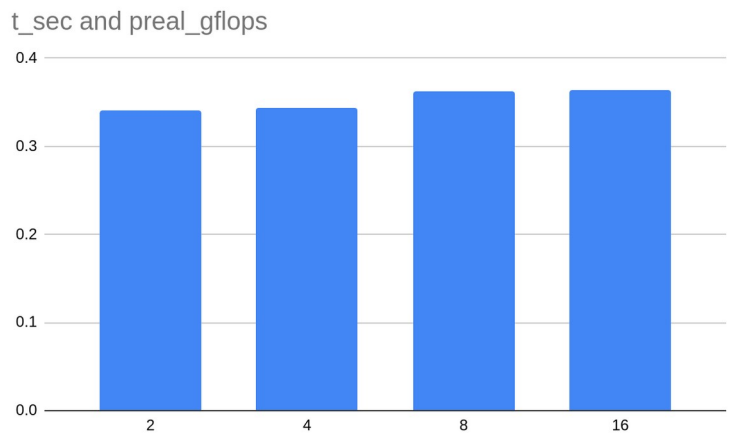
Таким образом, оптимальное значение $\mathbf{S^* = 8}$ (имеем P_{real} = 2.53197 GFLOP/s)

4. Произведена раскрутка внутренних циклов для умножения с буферизацией столбца и блочного умножения на различное число итераций M. Вычисление P_{real} в зависимости от числа итераций M для каждого значения S = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024; график и таблица зависимости для фиксированного N = 256 представлены ниже. (а) Диаграмма для буферизации столбца матрицы B при S = 1: оптимальное $\mathbf{M^* = 16}$ (имеем P_{real} = 2.66672 GFLOP/s)



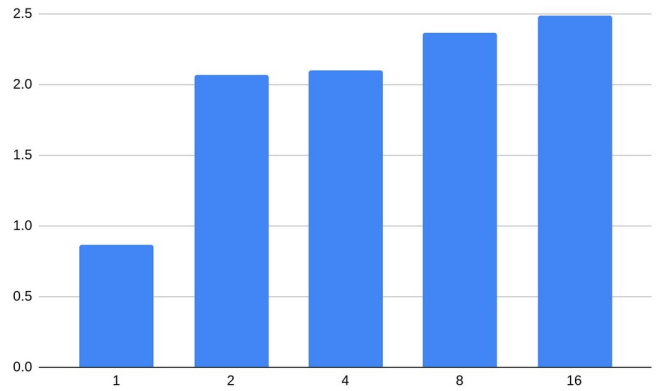
N	S	M	t_sec	preal_gflops
256	1	1	0.025004351	1.34194
256	1	2	0.015811557	2.12215
256	1	4	0.014492533	2.31529
256	1	8	0.013107427	2.55996
256	1	16	0.012582648	2.66672

(б) Диаграмма для блочного умножения при S = 1: оптимальное $\mathbf{M^* = 16}$ (имеем P_{real} = 0.363515 GFLOP/s)



N	S	M	t_sec	preal_gflops
256	1	1	0.115989133	0.289289
256	1	2	0.098448194	0.340833
256	1	4	0.097453441	0.344312
256	1	8	0.092606774	0.362332
256	1	16	0.092305527	0.363515

(в) Диаграмма для буферизации столбца матрицы B при S = 64: оптимальное $\mathbf{M^* = 16}$ (имеем P_{real} = 2.49091 GFLOP/s)



N	S	M	t_sec	preal_gflops
256	64	1	0.038731094	0.866344
256	64	2	0.016231725	2.06721
256	64	4	0.015981	2.09965
256	64	8	0.014153744	2.37071
256	64	16	0.013470728	2.49091

Почти во всех экспериментах оптимальным значением было значение $\mathbf{M^* = 16}$. Так что выбираем для дальнейшего сравнения именно его.

5. Ниже приведён скрипт для сравнения, N — степени двойки от 4 до 8192:

```
for N in 16 32 64 128 256 512 1024 2048; do
./lab2 --mode single --N $N --S 8 --M 16 --out results_N${N}.csv --repeats 3
done
```

Результат для N = 128 представлен ниже в виде таблицы:

mode	N	S	M	algo	t_sec	preal_gflops
single	128	8	16	classic	0.00794533	0.527896
single	128	8	16	transpose_total	0.006509108	0.644375
single	128	8	16	buffered	0.006636610999	0.631995
single	128	8	16	blocked	0.006904734999	0.607453

Результат для N = 2048 представлен ниже в виде таблицы:

mode	N	S	M	algo	t_sec	preal_gflops
single	2048	8	16	classic	50.21218085	0.342145
single	2048	8	16	transpose	8.181292781	2.0999
single	2048	8	16	buffered	9.234676567	1.86037
single	2048	8	16	blocked	11.34392617	1.51446

Таблица лучших методов для разных значений N:

N	1 место (max GFLOP/s)	GFLOP/s 1 места	2 место	GFLOP/s 2 места
16	transpose_mul_only	1.18966	transpose_total	1.09842
32	transpose_mul_only	1.07492	transpose_total	1.04217
64	buffered	0.748346	transpose_mul_only	0.64948
128	transpose_mul_only	0.657904	transpose_total	0.644375
256	blocked	2.92174	buffered	2.44445
512	blocked	2.84213	transpose_mul_only	2.43384
1024	blocked	2.70903	transpose_mul_only	2.26724
2048	transpose_mul_only	2.10656	transpose_total	2.0999

1) В классике элементы A читаются последовательно, а элементы B — с большим шагом по памяти (по столбцам), из-за чего кэш и предвыборка работают плохо; это и является основной причиной падения эффективности при росте N.

2) Транспонирование превращает обращение к B в последовательное чтение, поэтому после транспонирования обе матрицы обходятся “подряд” и кэш работает эффективнее. В таблице с итогами мы указали transpose_mul_only — без учёта времени на само транспонирование матрицы (например, для многоразового умножения это **наилучший** вариант)

3) Буферизация устраняет “прыжки” по памяти для B, копируя столбец в линейный массив, но добавляет накладные расходы копирования. Поэтому на средних N этот метод неплох, (например, для N=256 имеем 2.444 GFLOP/s)

4) Смысл блокирования — повышать локальность данных: для вычисления блока C нужны “полосы” A и B, и при удачном S это лучше укладывается в кэш. Но оптимальные S и M могут зависеть от N и от того, в какие уровни кэша попадают рабочие наборы данных. Для N=256, 512, 1024 этот метод лучший: **2.92**, **2.84**, **2.71** GFLOP/s соответственно, а на N=2048 он становится хуже чем transpose (транспонирование)

Выводы

- **Классический** алгоритм прост, но на больших матрицах часто самый медленный из-за неэффективного доступа к элементам матрицы B (плохая работа кэша).
- **Транспонирование** B обычно заметно ускоряет умножение, потому что чтение данных становится более последовательным. Минус — дополнительное время на само транспонирование; особенно выгодно, если матрица B используется многократно.
- **Буферизация столбца B** улучшает локальность доступа к B и ускоряет по сравнению с классикой, но добавляет накладные расходы на копирование столбцов, поэтому не всегда лучший вариант.
- **Блочное умножение** может давать наибольший выигрыш на средних и больших размерах, так как лучше использует кэш. При этом результат сильно зависит от правильно подобранного размера блока и степени раскрутки.
- **Раскрутка цикла** повышает производительность до некоторого оптимума, но слишком большая раскрутка может ухудшать скорость из-за разрастания кода.